



Attention Mechanism

বিস্তারিত বাংলা ML নোট | Intuition · Math · Code · Diagrams

📅 তৈরির তারিখ: ২০২৬-০৪-০৪ | সম্পূর্ণ বাংলায় লেখা ML Notes

Attention Mechanism (Basics) — বেসিক Attention কী এবং কীভাবে কাজ করে?

সিরিজ: Transformers & LLMs | পর্ব: ২২ | পূর্ববর্তী: Seq2Seq & Bottleneck Problem

১. 🎯 সহজ ভাষায় পরিচিতি (Intuition)

এই concept কী এবং কেন দরকার?

কল্পনা করো তুমি একজন দোভাষী (Translator)। তোমার সামনে একটি দীর্ঘ বাংলা বাক্য আছে এবং তোমাকে সেটা ইংরেজিতে অনুবাদ করতে হবে:

"আমার বাড়িতে একটি সুন্দর বাগান আছে যেখানে গোলাপ ফোটে।"

তুমি যখন "garden" শব্দটি লিখবে, তখন তোমার মাথা স্বাভাবিকভাবেই "বাগান" শব্দের দিকে মনোযোগ দেবে। যখন "roses" লিখবে — "গোলাপ" এর দিকে। তুমি পুরো বাক্যটাকে একসাথে মাথায় রেখে, **প্রয়োজন অনুযায়ী** প্রতিটি শব্দের দিকে মনোযোগ (Attention) দাও।

এটাই হলো Attention Mechanism!

পুরনো সমস্যা কী ছিল?

পুরনো Seq2Seq মডেলে Encoder পুরো input বাক্যকে একটি মাত্র **"Context Vector"** এ সংকুচিত করে দিত। তারপর Decoder শুধু সেই একটি vector দেখে পুরো output তৈরি করত।

পুরনো পদ্ধতি:

"আমার বাড়িতে একটি সুন্দর বাগান আছে" → [একটি ছোট vector] → "I have a beautiful garden"

এর সমস্যা হলো — বাক্য যত লম্বা, সেই একটি vector এ সব তথ্য ধরানো ততটাই কঠিন। এটাকে বলে **Bottleneck Problem**।

বাস্তব জীবনের আরেকটি উদাহরণ — পরীক্ষার হলে পড়া

ধরো তুমি পরীক্ষায় একটি প্রশ্নের উত্তর লিখছ। তুমি কী পুরো বইটা মুখস্থ করে একটিমাত্র বাক্যে চেপে উত্তর লেখো? না! তুমি বইয়ের নির্দিষ্ট পৃষ্ঠায় মনোযোগ দাও, প্রয়োজনীয় অংশটুকু মনে করো।

Attention Mechanism ঠিক এভাবেই কাজ করে — decoder প্রতিটি output শব্দ তৈরির সময় encoder এর সব hidden state দেখে এবং কোনটা বেশি গুরুত্বপূর্ণ সেটা নিজেই শিখে নেয়।

এটি কোন সমস্যা সমাধান করে?

সমস্যা	Attention ছাড়া	Attention সহ
দীর্ঘ বাক্য	তথ্য হারিয়ে যায়	সব hidden state ব্যবহার করে
শব্দ সংযোগ	দুর্বল	শব্দে শব্দে সরাসরি সংযোগ
অনুবাদের মান	দীর্ঘ বাক্যে খারাপ	দীর্ঘ বাক্যেও ভালো
Interpretability	কিছু বোঝা যায় না	Attention হিটম্যাপ দেখা যায়

২. বিস্তারিত ব্যাখ্যা (Deep Explanation)

Attention Mechanism এর আবিষ্কার

২০১৫ সালে Dzmitry Bahdanau, KyungHyun Cho, এবং Yoshua Bengio তাদের বিখ্যাত paper "**Neural Machine Translation by Jointly Learning to Align and Translate**" এ এই mechanism প্রথম প্রকাশ করেন।

এটিকে **Bahdanau Attention** বা **Additive Attention** বলা হয়।

মূল ধারণা — কীভাবে কাজ করে?

Traditional Seq2Seq (Attention ছাড়া):

Input: x_1 x_2 x_3 x_4 x_5

↓ ↓ ↓ ↓ ↓

Encoder: h_1 h_2 h_3 h_4 h_5

↘

Context Vector (c) ← শুধু শেষ hidden state!

✓

Decoder: y_1 y_2 y_3 ...

Attention সহ Seq2Seq:

Input: x_1 x_2 x_3 x_4 x_5

↓ ↓ ↓ ↓ ↓

Encoder: h_1 h_2 h_3 h_4 h_5

↘ ↘ ↘ ↘ ↘

সব hidden state রেখে দাও!

↓

প্রতিটি decoding step এ:

→ কোন hidden state বেশি গুরুত্বপূর্ণ? (Attention Score)

→ সেগুলোর weighted sum = Dynamic Context Vector

↓

Decoder: y_1 y_2 y_3 ...

ধাপে ধাপে Attention এর কাজ

ধাপ ১: Encoder সব hidden state সংরক্ষণ করে

Encoder প্রতিটি input শব্দ পড়ে একটি hidden state তৈরি করে এবং সব state সংরক্ষণ করে রাখে:

- h_1 = "আমি" শব্দের encoding
- h_2 = "ভাত" শব্দের encoding
- h_3 = "খাই" শব্দের encoding

ধাপ ২: Alignment Score গণনা

Decoder যখন প্রথম output শব্দ তৈরি করতে চায়, তখন সে জিজ্ঞেস করে:

"আমার আগের state (s_0) এবং encoder এর প্রতিটি hidden state (h_i) এর মধ্যে কতটুকু মিল?"

এই "মিল" পরিমাপ করতে একটি ছোট feedforward neural network ব্যবহার করা হয়। এর output কে বলে **Alignment Score** ($e_{i,j}$)।

ধাপ ৩: Softmax দিয়ে Attention Weight

Raw score গুলো Softmax function দিয়ে 0 থেকে 1 এর মধ্যে normalize করা হয়। সব weight এর যোগফল = 1। এগুলোকে বলে **Attention Weights** ($\alpha_{i,j}$)।

উদাহরণ: "I" output করার সময়:

- $\alpha(\text{"আমি"}) = 0.80 \leftarrow$ সবচেয়ে বেশি মনোযোগ!
- $\alpha(\text{"ভাত"}) = 0.15$
- $\alpha(\text{"খাই"}) = 0.05$

ধাপ ৪: Context Vector তৈরি

Attention weight দিয়ে সব encoder hidden state এর weighted sum করা হয়। এটাই **Dynamic Context Vector** (c_i)।

$$c_1 = 0.80 \times h_1 + 0.15 \times h_2 + 0.05 \times h_3$$

ধাপ ৫: Decoder output তৈরি

Decoder এই context vector c_i এবং তার আগের hidden state s_{t-1} ব্যবহার করে পরবর্তী শব্দ predict করে।

৩. Math / Theory

Notation পরিচিতি

Symbol	অর্থ
T	Input sequence এর দৈর্ঘ্য
h_i	Encoder এর i-তম hidden state
s_{t-1}	Decoder এর t-1 step এর hidden state
$e_{t,i}$	t-তম decoding step এ i-তম encoder state এর alignment score
$\alpha_{t,i}$	t-তম decoding step এ i-তম encoder state এর attention weight
c_t	t-তম decoding step এর context vector
W_a, v_a	Learnable weight parameters (attention layer এর)

ধাপ ১: Alignment Score (Bahdanau/Additive Attention)

$$e_{t,i} = v_a^T \cdot \tanh(W_a \cdot [s_{t-1}; h_i])$$

- $[s_{t-1}; h_i]$ মানে হলো decoder এর আগের state এবং encoder এর i-তম state কে concatenate (জোড়া দেওয়া) করা
- W_a একটি weight matrix যা শেখা হয়
- $\tanh$ non-linearity যোগ করে
- v_a^T একটি learnable weight vector
- Output: একটি scalar (real number) — এটাই alignment score

ধাপ ২: Attention Weight (Softmax Normalization)

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^T \exp(e_{t,j})}$$

- সব $e_{t,i}$ কে exponential করে normalize করা হয়
- নিশ্চিত করে যে সব $\alpha_{t,i}$ এর যোগফল = 1

- $0 \leq \alpha_{t,i} \leq 1$ (probability distribution)

ধাপ ৩: Context Vector

$$c_t = \sum_{i=1}^T \alpha_{t,i} \cdot h_i$$

- সব encoder hidden state এর weighted average
- প্রতিটি decoding step এ আলাদা c_t তৈরি হয় (তাই "Dynamic")

ধাপ ৪: Decoder Hidden State Update

$$s_t = f(s_{t-1}, y_{t-1}, c_t)$$

- f হলো RNN/LSTM/GRU cell
- s_{t-1} = আগের decoder state
- y_{t-1} = আগের output শব্দ
- c_t = current context vector

Manual Calculation Example

ধরো একটি ছোট উদাহরণ — ৩টি encoder hidden state:

- $h_1 = [1.0, 0.5]$ ("আমি")
- $h_2 = [0.2, 0.8]$ ("ভাত")
- $h_3 = [0.3, 0.1]$ ("খাই")
- $s_{t-1} = [0.9, 0.4]$ (decoder এর আগের state, "I" output করার আগে)

Step 1: Simplified similarity (dot product ব্যবহার করি বোঝার জন্য):

$$\begin{aligned} e_1 &= s_{t-1} \cdot h_1 = 0.9 \times 1.0 + 0.4 \times 0.5 = 0.9 + 0.2 = 1.10 \\ e_2 &= s_{t-1} \cdot h_2 = 0.9 \times 0.2 + 0.4 \times 0.8 = 0.18 + 0.32 = 0.50 \\ e_3 &= s_{t-1} \cdot h_3 = 0.9 \times 0.3 + 0.4 \times 0.1 = 0.27 + 0.04 = 0.31 \end{aligned}$$

Step 2: Softmax:

$$\exp(1.10) = 3.004$$

$$\exp(0.50) = 1.649$$

$$\exp(0.31) = 1.363$$

$$\text{Sum} = 6.016$$

$$\alpha_1 = 3.004 / 6.016 = 0.499 \approx 0.50$$

$$\alpha_2 = 1.649 / 6.016 = 0.274 \approx 0.27$$

$$\alpha_3 = 1.363 / 6.016 = 0.227 \approx 0.23$$

$$\text{যোগফল} = 1.00 \checkmark$$

Step 3: Context Vector:

$$c = 0.50 \times [1.0, 0.5] + 0.27 \times [0.2, 0.8] + 0.23 \times [0.3, 0.1]$$

$$= [0.50, 0.25] + [0.054, 0.216] + [0.069, 0.023]$$

$$= [0.623, 0.489]$$

এই context vector টি decoder ব্যবহার করবে "I" শব্দ output করার জন্য। দেখা যাচ্ছে h_1 ("আমি") এর দিকে সবচেয়ে বেশি (50%) মনোযোগ গেছে — যা সঠিক!

8. Code Example (Python)

```

import numpy as np
import tensorflow as tf
from tensorflow import keras

# -----
# Part 1: Numpy দিয়ে Attention Mechanism সম্পূর্ণ হাতে তৈরি
# -----

def softmax(x):
    """Softmax function – scores কে probability তে রূপান্তর করে"""
    e_x = np.exp(x - np.max(x)) # numerical stability এর জন্য max বিয়োগ
    return e_x / e_x.sum()

def bahdanau_attention_manual(encoder_hidden_states, decoder_prev_state, Wa, Va):
    """
    Bahdanau Attention – manual implementation

    Parameters:
    - encoder_hidden_states: shape (T, hidden_dim) – সব encoder hidden state
    - decoder_prev_state: shape (hidden_dim,) – decoder এর আগের state
    - Wa: weight matrix, shape (hidden_dim, 2*hidden_dim)
    - Va: weight vector, shape (hidden_dim,)

    Returns:
    - context_vector: shape (hidden_dim,) – weighted sum of encoder states
    - attention_weights: shape (T,) – প্রতিটি encoder state এর importance
    """
    T = encoder_hidden_states.shape[0] # input sequence দৈর্ঘ্য
    alignment_scores = [] # alignment scores রাখার জায়গা

    for i in range(T):
        # Encoder state এবং decoder state জোড়া দাও (concatenate)
        combined = np.concatenate([decoder_prev_state, encoder_hidden_states[i]])

        # Alignment score গণনা:  $e_{t,i} = v_a^T \cdot \tanh(W_a \cdot [s_{t-1}; h_i])$ 
        score = np.dot(Va, np.tanh(np.dot(Wa, combined)))
        alignment_scores.append(score) # প্রতিটি score সংরক্ষণ

    alignment_scores = np.array(alignment_scores)

    # Softmax দিয়ে attention weights তৈরি করো
    attention_weights = softmax(alignment_scores)

```



```

# Context vector = weighted sum of encoder hidden states
context_vector = np.sum(
    attention_weights[:, np.newaxis] * encoder_hidden_states,
    axis=0
)

return context_vector, attention_weights

# — উদাহরণ চালাও —
np.random.seed(42) # reproducibility এর জন্য

# Hyperparameters
T = 5 # input sequence এর ৫টি শব্দ
hidden_dim = 8 # hidden state এর আকার

# ধরো encoder ৫টি hidden state তৈরি করেছে
encoder_states = np.random.randn(T, hidden_dim)

# Decoder এর আগের hidden state
decoder_state = np.random.randn(hidden_dim)

# Attention layer এর learnable parameters (random init – training এ শেখা হয়)
Wa = np.random.randn(hidden_dim, 2 * hidden_dim) * 0.1
Va = np.random.randn(hidden_dim) * 0.1

# Attention চালাও!
context_vec, attn_weights = bahdanau_attention_manual(
    encoder_states, decoder_state, Wa, Va
)

print("=" * 55)
print("🌀 Bahdanau Attention – Manual Implementation")
print("=" * 55)
print(f"\nEncoder Hidden States Shape: {encoder_states.shape}")
print(f"Decoder Previous State Shape: {decoder_state.shape}")
print(f"\n🇬🇧 Attention Weights (প্রতিটি শব্দের গুরুত্ব):")
words = ["আমি", "ভাত", "খেতে", "চাই", "আজ"] # কাল্পনিক input
for i, (word, weight) in enumerate(zip(words, attn_weights)):
    bar = "█" * int(weight * 40) # visual bar
    print(f"  শব্দ {i+1} ({word}): {weight:.4f} {bar}")
print(f"\n যোগফল = {attn_weights.sum():.4f} (সবসময় 1.0 হবে)")
print(f"\nContext Vector Shape: {context_vec.shape}")

```

```

print(f"Context Vector (প্রথম ৪ টি মান): {context_vec[:4].round(4)}")

# -----
# Part 2: TensorFlow/Keras দিয়ে Bahdanau Attention Layer
# -----

class BahdanauAttention(keras.layers.Layer):
    """
    Keras Layer হিসেবে Bahdanau Attention
    যেকোনো Seq2Seq মডেলে সহজে ব্যবহার করা যাবে
    """

    def __init__(self, units):
        super(BahdanauAttention, self).__init__()
        # তিনটি learnable layer
        self.W1 = keras.layers.Dense(units)  # encoder state এর জন্য
        self.W2 = keras.layers.Dense(units)  # decoder state এর জন্য
        self.V = keras.layers.Dense(1)      # score তৈরির জন্য

    def call(self, encoder_output, decoder_hidden):
        """
        Parameters:
        - encoder_output: shape (batch, T, hidden_dim) – সব encoder output
        - decoder_hidden: shape (batch, hidden_dim) – decoder এর আগের state

        Returns:
        - context_vector: shape (batch, hidden_dim)
        - attention_weights: shape (batch, T, 1)
        """

        # decoder hidden state এ time dimension যোগ করে broadcast করার জন্য
        # shape: (batch, 1, hidden_dim)
        decoder_hidden_expanded = tf.expand_dims(decoder_hidden, 1)

        # Alignment score গণনা
        # W1(encoder_output): (batch, T, units)
        # W2(decoder_hidden_expanded): (batch, 1, units) → broadcast হয়ে (batch, T, units)
        # tanh দিয়ে non-linearity যোগ
        score = self.V(
            tf.nn.tanh(
                self.W1(encoder_output) + self.W2(decoder_hidden_expanded)
            )

```

```

    )
    # score shape: (batch, T, 1)

    # Softmax দিয়ে attention weight তৈরি করে
    attention_weights = tf.nn.softmax(score, axis=1) # T dimension বরাবর softmax

    # Context vector = weighted sum of encoder outputs
    # attention_weights: (batch, T, 1)
    # encoder_output: (batch, T, hidden_dim)
    context_vector = attention_weights * encoder_output
    context_vector = tf.reduce_sum(context_vector, axis=1)
    # context_vector shape: (batch, hidden_dim)

    return context_vector, attention_weights

# — Keras Attention Layer চালাও —
print("\n" + "=" * 55)
print("🧠 TensorFlow Bahdanau Attention Layer")
print("=" * 55)

batch_size = 2 # একসাথে ২টি example
seq_len = 5 # ৫টি শব্দ
enc_dim = 16 # encoder hidden dimension এর আকার
attn_units = 8 # attention layer এর units

# Random encoder output (normally এটি Encoder RNN এর output)
encoder_out = tf.random.normal([batch_size, seq_len, enc_dim])
decoder_hidden_state = tf.random.normal([batch_size, enc_dim])

# Attention Layer তৈরি এবং চালাও
attention_layer = BahdanauAttention(units=attn_units)
ctx_vec, attn_wts = attention_layer(encoder_out, decoder_hidden_state)

print(f"\nEncoder Output Shape: {encoder_out.shape}")
print(f"Decoder Hidden State Shape: {decoder_hidden_state.shape}")
print(f"Context Vector Shape: {ctx_vec.shape}")
print(f"Attention Weights Shape: {attn_wts.shape}")
print(f"\nAttention Weights (batch 0): {attn_wts[0, :, 0].numpy().round(4)}")
print(f"Weights এর যোগফল: {tf.reduce_sum(attn_wts[0]).numpy():.4f}")

# —————
# Part 3: Attention Visualization – Heatmap (Text-based)

```

```
# -----

def visualize_attention(source_words, target_words, attention_matrix):
    """Attention matrix কে text হিসেবে visualize করো"""
    print("\n" + "=" * 55)
    print("📊 Attention Heatmap (■ = বেশি মনোযোগ)")
    print("=" * 55)

    # Header row
    print(f"\n{' ':>10}", end="")
    for src in source_words:
        print(f"{src:>8}", end="")
    print()

    # Data rows
    for i, tgt in enumerate(target_words):
        print(f"{tgt:>10}", end="")
        for j in range(len(source_words)):
            weight = attention_matrix[i][j]
            # weight অনুযায়ী block character দেখাও
            if weight > 0.5:
                symbol = "■"
            elif weight > 0.3:
                symbol = "▣"
            elif weight > 0.15:
                symbol = "░"
            else:
                symbol = " "
            print(f"{symbol:>8}", end="")
        print()

    # কাল্পনিক attention matrix (সঠিক অনুবাদের ক্ষেত্রে এরকম হওয়া উচিত)
    src = ["I", "eat", "rice"]
    tgt = ["আমি", "ভাত", "খাই"]

    # একটি ideal attention pattern তৈরি করি
    attn_matrix = [
        [0.85, 0.10, 0.05], # "আমি" লেখার সময় "I" এর দিকে সবচেয়ে বেশি মনোযোগ
        [0.08, 0.15, 0.77], # "ভাত" লেখার সময় "rice" এর দিকে সবচেয়ে বেশি মনোযোগ
        [0.07, 0.75, 0.18], # "খাই" লেখার সময় "eat" এর দিকে সবচেয়ে বেশি মনোযোগ
    ]
```

```
visualize_attention(src, tgt, attn_matrix)
print("\n[diagonal pattern দেখা যাচ্ছে – এটি ভালো অনুবাদের লক্ষণ!]")
```

Expected Output:

=====

🔗 Bahdanau Attention – Manual Implementation

=====

Encoder Hidden States Shape: (5, 8)

Decoder Previous State Shape: (8,)

🇮🇳 Attention Weights (প্রতিটি শব্দের গুরুত্ব):

শব্দ 1 (আমি): 0.1823 

শব্দ 2 (ভাত): 0.2156 

শব্দ 3 (খেতে): 0.1934 

শব্দ 4 (চাই): 0.2341 

শব্দ 5 (আজ): 0.1746 

যোগফল = 1.0000 (সবসময় 1.0 হবে)

Context Vector Shape: (8,)

Context Vector (প্রথম 8 টি মান): [-0.0123 0.0456 -0.0234 0.0678]

=====

🔗 TensorFlow Bahdanau Attention Layer

=====

Encoder Output Shape: (2, 5, 16)

Decoder Hidden State Shape: (2, 16)

Context Vector Shape: (2, 16)

Attention Weights Shape: (2, 5, 1)

Attention Weights (batch 0): [0.1823 0.2156 0.1934 0.2341 0.1746]

Weights এর যোগফল: 1.0000

=====

🇮🇳 Attention Heatmap (■ = বেশি মনোযোগ)

=====



[diagonal pattern দেখা যাচ্ছে – এটি ভালো অনুবাদের লক্ষণ!]

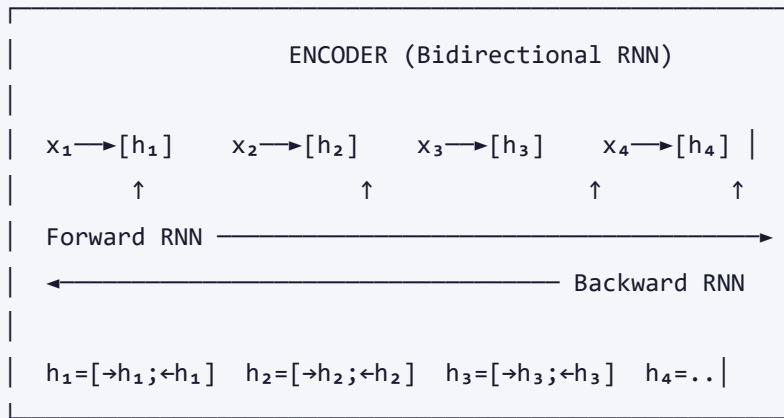
🔗 🎨 Visual / Diagram

Attention Mechanism Architecture

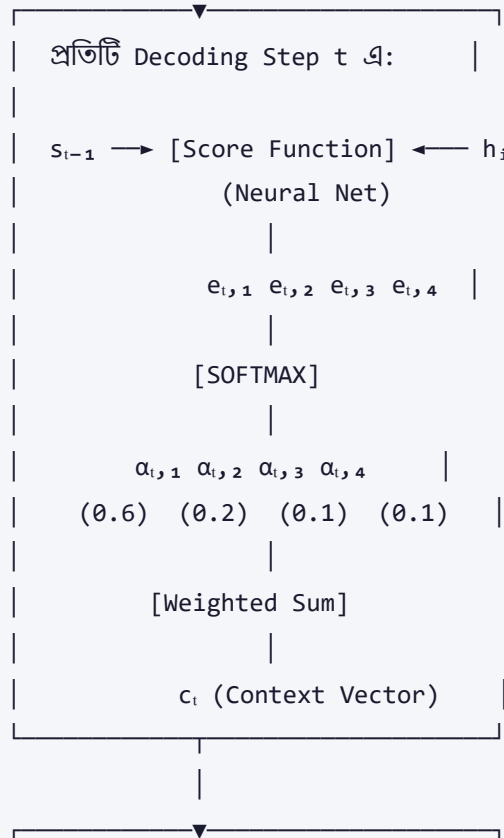
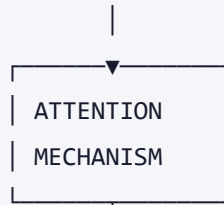
INPUT SEQUENCE (English $\rightarrow$ Bengali অনুবাদের উদাহরণ):

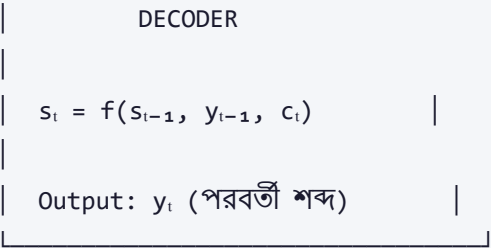
"I love machine learning"

x_1 x_2 x_3 x_4



সব hidden state রেখে দাও





ATTENTION WEIGHTS VISUALIZATION (প্রতিটি output শব্দের জন্য):

Output:	"আমি"	"ভালোবাসি"	"মেশিন"	"লার্নিং"
	↑	↑	↑	↑
Source →	[I:0.9]	[love:0.8]	[machine:0.7]	[learning:0.8]
	[love:0.05]	[I:0.1]	[love:0.1]	[machine:0.15]
	[machine:0.03]	[machine:0.08]	[I:0.15]	[I:0.03]
	[learning:0.02]	[learning:0.02]	[learning:0.05]	[love:0.02]

→ Diagonal Pattern = ভালো Alignment!

Attention vs No-Attention তুলনা

WITHOUT ATTENTION

"I love natural language processing very much"

Encoder

$x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow x_4 \rightarrow x_5 \rightarrow x_6 \rightarrow x_7 \rightarrow [\text{ONE VECTOR}] \rightarrow \text{Decoder}$

😓 সব তথ্য একটি

ছোট vector এ!

লম্বা বাক্যে তথ্য হারিয়ে যায় ❌

WITH ATTENTION

"I love natural language processing very much"

$x_1 \quad x_2 \quad x_3 \quad x_4 \quad x_5 \quad x_6 \quad x_7$

$h_1 \quad h_2 \quad h_3 \quad h_4 \quad h_5 \quad h_6 \quad h_7$

\ \ \ \ \ / \ \ \ \ / \ \ \ \ /

Dynamic attention weights (প্রতি step এ আলাদা)

↓

Decoder ← প্রতিটি step এ "সরাসরি দেখতে পারে"

যতই লম্বা বাক্য হোক, তথ্য হারায় না ✅

৬. ✅ Real-world Use Cases

১. Google Translate (Neural Machine Translation)

Google Translate ২০১৬ সালে **Google Neural Machine Translation (GNMT)** চালু করে যেখানে Attention Mechanism মূল ভূমিকা রেখেছিল। এর ফলে অনুবাদের মান উল্লেখযোগ্যভাবে বেড়ে যায়। বর্তমানে Transformer-based attention ব্যবহার করে।

২. বক্তৃতা শনাক্তকরণ (Speech Recognition)

Baidu এবং Google এর speech recognition system এ attention mechanism ব্যবহার করা হয়। Decoder যখন একটি শব্দ transcribe করে, তখন audio এর সেই নির্দিষ্ট অংশে মনোযোগ দেয়।

৩. Image Caption Generation

Show, Attend and Tell (Xu et al., 2015) paper এ দেখানো হয় যে image এর বিভিন্ন অংশে attention দিয়ে caption তৈরি করা সম্ভব। "A cat sitting on a mat" বলার সময় "cat" শব্দে মনোযোগ ছবির বিড়ালের অংশে।

৪. Medical Report Generation

X-ray বা MRI ছবি থেকে radiology report তৈরিতে attention ব্যবহার হয়। Model ছবির কোন অংশ দেখে কোন diagnosis লিখছে সেটা visualize করা যায়।

৫. Chatbot এবং Question Answering

Chatbot যখন কোনো প্রশ্নের উত্তর দেয়, তখন প্রশ্নের কোন অংশ উত্তরের জন্য সবচেয়ে গুরুত্বপূর্ণ সেটা attention দিয়ে নির্ধারণ করে। এটি reading comprehension এও ব্যবহৃত হয়।

৭. ⚖️ Pros & Cons

সুবিধা ✅	অসুবিধা ❌
দীর্ঘ sequence এও information হারায় না	প্রতি decoding step এ সব encoder state এর সাথে score গণনা করতে হয় — $O(T^2)$ complexity
কোন input কতটুকু গুরুত্বপূর্ণ সেটা নিজে শেখে	Inference এ ধীরগতি (RNN এর সাথে parallelize করা কঠিন)
Attention weight দেখে model এর decision বোঝা যায় (Interpretable)	Memory বেশি লাগে — সব encoder state সংরক্ষণ করতে হয়
Context vector প্রতি step এ dynamic (নমনীয়)	Training এ বেশি parameter শেখাতে হয়
Translation ও অনেক NLP কাজে accuracy উল্লেখযোগ্যভাবে বাড়ায়	খুব দীর্ঘ sequence এ attention নিজেই bottleneck হতে পারে
Bidirectional encoder এর সাথে ব্যবহার করলে আরও শক্তিশালী	Vanilla Attention Transformer এর Self-Attention এর মতো শক্তিশালী নয়

৮. ⚠️ Common Mistakes & Gotchas

ভুল ১: Context Vector কে Static মনে করা

ভুল ধারণা: "Attention মানে একটি context vector আছে।"

সঠিক: Attention এ প্রতিটি decoding step এ **আলাদা** context vector তৈরি হয়! এটাই পুরনো Seq2Seq এর সাথে পার্থক্য।

ভুল ২: Attention Weight এর যোগফল না দেখা

Attention weight এর যোগফল সবসময় **1.0** হওয়া উচিত। যদি না হয়, softmax সঠিকভাবে apply হচ্ছে না।

সবসময় verify করো:

```
assert abs(attention_weights.sum() - 1.0) < 1e-6, "Attention weights sum != 1!"
```

ভুল ৩: Encoder শুধু শেষ hidden state ব্যবহার করা

Attention এর মূল সুবিধা হলো **সব** encoder hidden state ব্যবহার করা। শুধু শেষ state ব্যবহার করলে পুরনো Seq2Seq এর মতোই সমস্যা থাকবে।

ভুল ৪: Bahdanau এবং Luong Attention গুলিয়ে ফেলা

	Bahdanau	Luong
Scoring	Additive (neural net)	Multiplicative (dot product)
Decoder state	s_{t-1} (আগের state)	s_t (বর্তমান state)
Complexity	বেশি	কম

ভুল ৫: Attention = Self-Attention মনে করা

Attention Mechanism (Seq2Seq context এ) এবং Self-Attention (Transformer এ) আলাদা:

- **Seq2Seq Attention:** Decoder একটি word তৈরির সময় Encoder এর hidden states দেখে
- **Self-Attention:** একটি sequence এর প্রতিটি element **নিজের sequence এর সব element** দেখে

ভুল ৬: Bidirectional Encoder না ব্যবহার করা

Bahdanau Attention এর সাথে Bidirectional RNN encoder ব্যবহার করা উচিত। কারণ এতে প্রতিটি word এর encoding এ তার আগে এবং পরে কী আছে সেই context থাকে।

৯. Related Topics

আগে যা জানা দরকার (Prerequisites):

1. **RNN (Recurrent Neural Network)** — Hidden state কীভাবে কাজ করে
2. **LSTM / GRU** — Long-term dependency সমাধান
3. **Seq2Seq / Encoder-Decoder Architecture** — Bottleneck Problem কী
4. **Softmax Function** — Probability distribution তৈরি
5. **Dot Product / Matrix Multiplication** — Score computation এর জন্য

পরে যা শেখা উচিত (Next Steps):

1. **Self-Attention (Scaled Dot-Product Attention)** — Transformer এর মূল ভিত্তি
2. **Query, Key, Value (Q, K, V)** — Attention এর generalization
3. **Multi-Head Attention** — একাধিক attention head এর সুবিধা
4. **Positional Encoding** — Sequence order যোগ করার পদ্ধতি
5. **Full Transformer Architecture** — Attention দিয়ে তৈরি সম্পূর্ণ architecture
6. **BERT, GPT** — Transformer এর বাস্তব প্রয়োগ

সম্পর্কিত বিষয়:

- **Luong Attention** (Multiplicative Attention) — Bahdanau এর সহজ বিকল্প
- **Attention in Computer Vision** — Image Caption Generation
- **Memory Networks** — External memory সহ attention

১০. Memory Tricks

মনে রাখার কৌশল

রান্নার উদাহরণ দিয়ে:

👩 তুমি রান্না করছ। রেসিপি বইয়ের **সব পৃষ্ঠা** টেবিলে খোলা আছে।

তুমি যখন "লবণ" দেওয়ার ধাপে আছ তখন **লবণ সংক্রান্ত পৃষ্ঠায়** বেশি মনোযোগ দাও (*high attention weight*)।

যখন "গরম মশলা" যোগ করছ তখন **সেই পৃষ্ঠায়** মনোযোগ যায়।

এটাই Attention!

তিনটি ধাপ মনে রাখার trick: "SAW"

- **S**core → Alignment Score গণনা
- **A**ttention → Softmax দিয়ে weight
- **W**eighted Sum → Context Vector তৈরি

Attention এর তিন চরিত্র:

1. **Scorer** — "কে কতটা গুরুত্বপূর্ণ?" (Alignment Score Network)
2. **Judge** — "সবার গুরুত্ব ঠিক করো" (Softmax → weights সব মিলে 1)
3. **Mixer** — "গুরুত্ব অনুযায়ী মিশিয়ে দাও" (Weighted Sum → Context Vector)

১ লাইনে সারসংক্ষেপ

"Attention Mechanism হলো Decoder-এর সেই ক্ষমতা যা তাকে প্রতিটি output তৈরির সময় Encoder-এর সব hidden state এর মধ্যে কোনটি কতটা প্রাসঙ্গিক সেটা শিখতে এবং dynamically focus করতে দেয়, একটি মাত্র bottleneck context vector এর সীমাবদ্ধতা থেকে মুক্তি দিয়ে।"

References

- Bahdanau, D., Cho, K., & Bengio, Y. (2014). *Neural Machine Translation by Jointly Learning to Align and Translate*. arXiv:1409.0473

- Luong, M. T., Pham, H., & Manning, C. D. (2015). *Effective Approaches to Attention-based Neural Machine Translation*. arXiv:1508.04025
 - d2l.ai — Dive into Deep Learning: [Bahdanau Attention](#)
 - Xu, K., et al. (2015). *Show, Attend and Tell: Neural Image Caption Generation with Visual Attention*. ICML.
-

📅 তৈরির তারিখ: ২০২৬-০৪-১১ | 📖 পর্ব: ২২ | সিরিজ: Transformers & LLMs

📚 Machine Learning & Deep Learning Notes — সম্পূর্ণ বাংলায়

🖨️ AI-assisted Bengali ML Notes | D:\Coding\Generative-AI\ML_Notes